

Disciplina: **ENE0025 – Protocolos de Transporte e Roteamento**

Curso: **Engenharia de Redes de Comunicação**

Instituição: **Universidade de Brasília (UnB)**

Departamento: **Engenharia Elétrica**

Professor Responsável: **Prof. Dr. Laerte Peotta de Melo**

Relatório do Laboratório 10B

Identificação

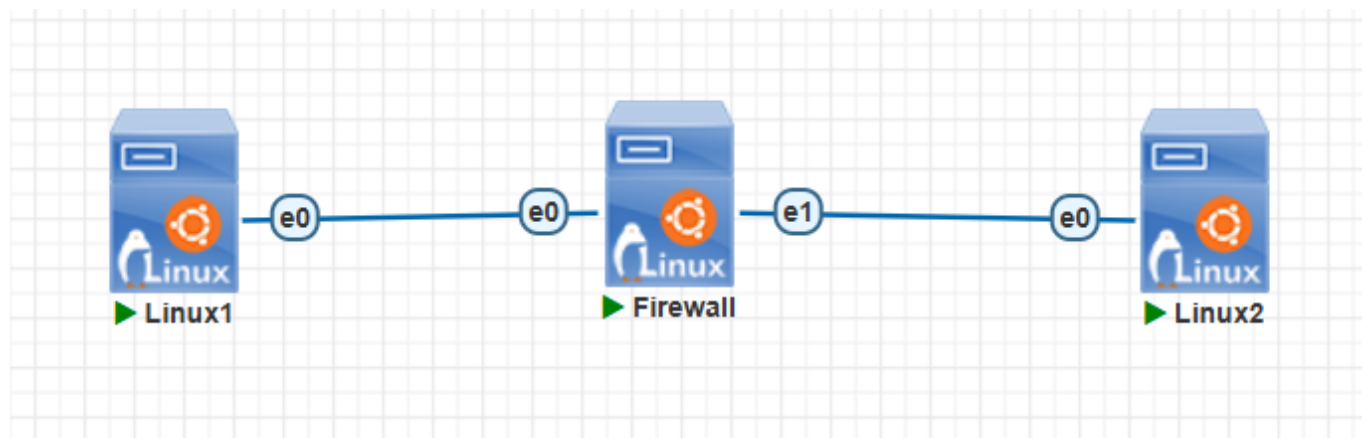
- Nome: **Artur Kohara Guerra**
 - Matrícula: **231025181**
 - Turma: **01**
-

Objetivo

Implementar um **firewall stateful** em uma máquina Linux no PNetLab , reutilizando a topologia do **Laboratório 10**, com regras baseadas em **estado de conexão** para permitir automaticamente o tráfego de retorno de sessões já autorizadas.

Topologia do Laboratório

A topologia utilizada é a mesma do Laboratório 10.



Procedimentos

Verificação da configuração IP

- Cliente 1

```
gns3@box:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 9a:2b:52:56:82:5c brd ff:ff:ff:ff:ff:ff
3: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
4: ip_vti0@NONE: <NOARP> mtu 1364 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 50:e8:ca:06:03:00 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.10/24 scope global eth0
        valid_lft forever preferred_lft forever
```

- Cliente 2

```
gns3@box:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 22:11:18:a7:a9:7c brd ff:ff:ff:ff:ff:ff
3: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
4: ip_vti0@NONE: <NOARP> mtu 1364 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 50:08:c5:06:04:00 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.10/24 scope global eth0
        valid_lft forever preferred_lft forever
```

- Firewall

```
user@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 50:7f:b5:05:fc:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.10.1/24 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::527f:b5ff:fe05:fc00/64 scope link
        valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 50:7f:b5:05:fc:01 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 192.168.20.1/24 scope global ens4
        valid_lft forever preferred_lft forever
    inet6 fe80::527f:b5ff:fe05:fc01/64 scope link
        valid_lft forever preferred_lft forever
```

```
sudo iptables -F
sudo iptables -X
sudo iptables -Z
sudo iptables -P FORWARD DROP
```

Regra central do firewall statefull

```
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Essa regra permite:

- pacotes que pertencem a conexões já aceitas;
- pacotes relacionados a conexões existentes.

Regras de novas conexões iniciadas pelo Cliente 1

Permitir ICMP iniciado pelo Cliente 1

```
sudo iptables -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -p icmp -m conntrack --ctstate NEW -j ACCEPT
```

Permitir HTTP iniciado pelo Cliente 1

```
sudo iptables -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -p tcp --dport 80 -m conntrack --ctstate NEW -j ACCEPT
```

Verificação das regras

```
user@ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source               destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source               destination ctstate
  0    0 ACCEPT    0    --  *      *       0.0.0.0/0           0.0.0.0/0          ctstate RELATED,ESTABLISHED
  0    0 ACCEPT    1    --  *      *       192.168.10.10       192.168.20.10      ctstate NEW
  0    0 ACCEPT    6    --  *      *       192.168.10.10       192.168.20.10      tcp dpt:80 ctstate NEW

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source               destination
```

```
user@ubuntu:~$ sudo iptables -S
-P INPUT ACCEPT
-P FORWARD DROP
-P OUTPUT ACCEPT
-A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -s 192.168.10.10/32 -d 192.168.20.10/32 -p icmp -m conntrack --ctstate NEW -j ACCEPT
-A FORWARD -s 192.168.10.10/32 -d 192.168.20.10/32 -p tcp -m tcp --dport 80 -m conntrack --ctstate NEW -j ACCEPT
```

Testes práticos

Teste de icmp

- No Linux Cliente 1:

```
gns3@box:~$ ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=63 time=4.089 ms
64 bytes from 192.168.20.10: seq=1 ttl=63 time=2.429 ms
64 bytes from 192.168.20.10: seq=2 ttl=63 time=3.974 ms
64 bytes from 192.168.20.10: seq=3 ttl=63 time=1.647 ms
64 bytes from 192.168.20.10: seq=4 ttl=63 time=1.370 ms
64 bytes from 192.168.20.10: seq=5 ttl=63 time=1.314 ms
^C
--- 192.168.20.10 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 1.314/2.470/4.089 ms
```

Nota-se que essa conexão funcionou, como esperado.

- No Linux Cliente 2:

```
gns3@box:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10): 56 data bytes
^C
--- 192.168.10.10 ping statistics ---
8 packets transmitted, 0 packets received, 100% packet loss
```

Observa-se que essa conexão falhou, como esperado.

Teste de HTTP

No Cliente 2, utilizou-se o Netcat para simular um servidor HTTP na porta 80:

```
gns3@box:~$ sudo nc -l -p 80
GET / HTTP/1.1
Host: 192.168.20.10
User-Agent: Wget
Connection: close
```

No Linux Cliente 1, testou-se o acesso:

```
gns3@box:~$ wget -O- http://192.168.20.10
Connecting to 192.168.20.10 (192.168.20.10:80)
```

Dessa maneira, percebe-se que o teste foi um sucesso, pois a conexão foi estabelecida.

Teste de nova conexão iniciada pelo Cliente 2

```
gns3@box:~$ nc -vz 192.168.10.10 80
nc: 192.168.10.10 (192.168.10.10:80): Connection timed out
```

Verifica-se que a conexão falhou, como esperado.

Teste de Telnet não permitido

```
gns3@box:~$ nc -vz 192.168.20.10 23
nc: 192.168.20.10 (192.168.20.10:23): Connection timed out
```

Percebe-se que a conexão falhou, como esperado.

Comparação entre firewall de pacotes (Laboratório 10) e firewall stateful (Laboratório 10B)

No Laboratório 10 foi implementado um firewall de pacotes utilizando o **iptables**, no qual cada pacote é analisado individualmente com base em critérios como endereço IP, protocolo e porta. Nesse modelo, o firewall não possui conhecimento sobre o estado das conexões, sendo necessário criar regras explícitas tanto para o tráfego de ida quanto para o tráfego de retorno.

Já no Laboratório 10B foi implementado um firewall stateful utilizando o mecanismo de rastreamento de conexões (**conntrack**). Nesse caso, o firewall passou a acompanhar o estado das conexões, permitindo automaticamente os pacotes pertencentes a sessões já estabelecidas por meio da regra **ESTABLISHED,RELATED**. Como consequência, houve uma redução significativa na quantidade de regras necessárias e uma simplificação da política de segurança.

Teste	Laboratório 10 - Filtro de pacotes	Laboratório 10B - Stateful
Ping iniciado pelo Cliente 1	Funciona devido às regras ICMP explícitas de ida e volta.	Funciona porque a conexão é iniciada pelo Cliente 1 e o retorno é permitido automaticamente por ESTABLISHED .
Retorno da comunicação	Necessita regra explícita para o tráfego de retorno.	É permitido automaticamente pela regra ESTABLISHED,RELATED .
HTTP Cliente 1 → Cliente 2	Funciona, mas exige regra para a conexão de ida e outra para o retorno.	Funciona com apenas uma regra para a conexão nova; o retorno é permitido automaticamente.
Nova conexão iniciada pelo Cliente 2	Foi bloqueada, pois não existe regra permitindo essa conexão.	Não funciona, pois apenas conexões NEW iniciadas pelo Cliente 1 são permitidas.

Teste	Laboratório 10 - Filtro de pacotes	Laboratório 10B - Stateful
Quantidade de regras	Maior, pois é necessário criar regras para ida e volta de cada serviço.	Menor, pois uma única regra ESTABLISHED,RELATED trata automaticamente o retorno das conexões.
Facilidade de administração	Menor, devido à necessidade de manter várias regras explícitas.	Maior, pois a política fica mais simples, organizada e escalável.

Questões para análise

- 1. O que diferencia um firewall stateful de um firewall de pacotes simples?

Um firewall de pacotes simples analisa cada pacote individualmente com base em informações no header, como endereço IP, protocolo e porta, sem considerar o contexto da comunicação. Já um firewall stateful acompanha o estado das conexões e consegue identificar se um pacote pertence a uma sessão já estabelecida, permitindo um controle mais inteligente do tráfego.

- 2. Qual a função de `-m conntrack --ctstate ESTABLISHED,RELATED`?

Essa regra utiliza o mecanismo de rastreamento de conexões do Linux para permitir pacotes pertencentes a conexões já estabelecidas (**ESTABLISHED**) ou relacionadas a elas (**RELATED**). Dessa forma, o tráfego de resposta de uma comunicação autorizada é aceito automaticamente sem necessidade de regras adicionais.

- 3. Por que o retorno da conexão HTTP não precisou de regra explícita no sentido inverso?

Isso ocorreu, pois o firewall stateful reconheceu que os pacotes de resposta pertenciam a uma conexão já estabelecida. A regra **ESTABLISHED,RELATED** permitiu automaticamente esse tráfego de retorno, eliminando a necessidade de criar uma regra específica para o fluxo inverso.

- 4. O que caracteriza um pacote no estado **NEW**?

Um pacote no estado **NEW** é aquele que inicia uma nova comunicação ou conexão. No contexto do laboratório, os pacotes enviados pelo Cliente 1 para iniciar uma sessão HTTP ou ICMP foram classificados como **NEW**, sendo avaliados pelas regras que permitem novas conexões.

- 5. Qual a principal vantagem de usar regras stateful em relação ao Laboratório 10?

A principal vantagem é a redução da quantidade de regras necessárias. Como o firewall acompanha o estado das conexões, o tráfego de retorno é tratado automaticamente, simplificando a configuração da política de segurança.

- 6. Por que o Cliente 2 não conseguiu iniciar novas conexões para o Cliente 1?

Porque as regras do firewall permitiam apenas conexões classificadas como **NEW** originadas pelo Cliente 1. Como não existia nenhuma regra autorizando novas conexões iniciadas pelo Cliente 2, os pacotes foram bloqueados pela política padrão da cadeia **FORWARD**, configurada como **DROP**.

- 7. O que mudou na quantidade e na lógica das regras entre Laboratório 10 e Laboratório 10B?

No Laboratório 10 foi necessário criar regras separadas para o tráfego de ida e de volta de cada serviço permitido. Já no Laboratório 10B uma única regra para conexões ESTABLISHED,RELATED passou a tratar automaticamente o retorno das comunicações, reduzindo a quantidade de regras.

- 8. Em que tipo de ambiente um firewall stateful tende a ser mais adequado?

Firewalls stateful são mais adequados para ambientes corporativos, redes empresariais, data centers e redes com grande quantidade de usuários e conexões simultâneas. Nesses cenários, o acompanhamento do estado das conexões aumenta a eficiência da filtragem e simplifica a administração das políticas de segurança.

- 9. O firewall stateful elimina a necessidade de política de bloqueio padrão? Explique.

Não. Mesmo em um firewall stateful, a política de bloqueio padrão continua sendo importante para garantir que todo tráfego não autorizado seja descartado. O rastreamento de conexões apenas facilita o tratamento das comunicações válidas, mas ainda é necessário definir quais novas conexões podem ser iniciadas e bloquear o restante do tráfego.

- 10. O que a atividade comparativa mostrou de forma mais clara sobre a diferença entre os dois modelos?

A atividade comparativa mostrou que o firewall de pacotes exige regras explícitas para cada direção da comunicação, enquanto o firewall stateful utiliza o rastreamento de conexões para permitir automaticamente o tráfego de retorno de sessões autorizadas. Isso reduz a quantidade de regras necessárias, simplifica a configuração e torna a política de segurança mais eficiente e fácil de administrar.

Conclusão

Neste laboratório foi possível implementar e analisar o funcionamento de um firewall stateful utilizando o `iptables` e o mecanismo de rastreamento de conexões (`conntrack`) em uma máquina Linux atuando como firewall entre duas redes distintas. Por meio dos testes realizados, observou-se que o firewall passou a reconhecer conexões já estabelecidas, permitindo automaticamente o tráfego de retorno sem a necessidade de regras explícitas para cada direção da comunicação. A comparação com o Laboratório 10 evidenciou que o modelo stateful reduz a quantidade de regras necessárias, simplifica a administração da política de segurança e torna o controle de tráfego mais eficiente. Dessa forma, conclui-se que os firewalls stateful oferecem uma solução mais prática e adequada para ambientes reais, mantendo a segurança da rede ao mesmo tempo em que facilitam a gestão das comunicações autorizadas.